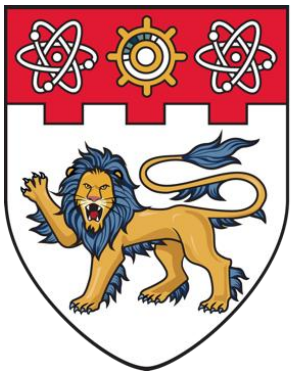


NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

EE6483: ARTIFICIAL INTELLIGENCE & DATA MINING

Dogs vs. Cats Classification and CIFAR-10 Imbalance Study

(Mini project)

Author:

SCHOOL OF ELECTRICAL AND ELECTRONIC ENGINEERING

Date of submission:

15 April 2026

Table of Contents

Personal Contribution Table

Name	ID	Email	Contribution Summary
			Model implementation, training pipeline setup, and main experiments, and final writing/formatting
			Literature survey, report integration, extension experiments, and final writing/formatting
			Result analysis, figure/table organization, final writing/formatting, and proofreading

Abstract

This report presents a reproducible transfer-learning pipeline for image classification project. For the primary Dogs vs. Cats task, the project formulates the problem as supervised closed-set binary classification and evaluates four practical backbones: ResNet18, ResNet34, MobileNetV3-Small, and a lightweight SmallCNN trained from scratch. The implementation uses ImageNet-pretrained models whenever available, applies moderate geometric and photometric augmentation, fixes the random seed for reproducibility, and exports a competition-ready submission.csv for the unlabeled test set. Across the archived training curves, the strongest model is ResNet34 with an approximate best validation accuracy of 99.2%, while MobileNetV3 -Small achieves a competitive result with far fewer parameters. An augmentation ablation further shows that the learned classifier is more stable when the training pipeline includes randomized transforms.

The report also extends the same design philosophy to the 10-class CIFAR-10 benchmark. A fine-tuned ResNet18 reaches about 96.8% validation accuracy in the saved run, showing that the pipeline generalizes well beyond binary classification. To study data imbalance, the CIFAR-10 training set is deliberately skewed by reducing classes 0, 1, and 8 to 20% of their original size. Among the tested remedies, weighted sampling obtains the best validation accuracy (94.04%), slightly outperforming class-weighted cross-entropy and focal loss. Overall, the experiments demonstrate that a carefully tuned pretrained residual backbone is a strong baseline for medium-scale image classification, while sampling-based imbalance mitigation provides a simple and effective improvement under skewed supervision.

1. Literature Survey and Problem Framing

1.1 Problem definition and learning settings

Image classification maps an input image x to a semantic label y drawn from a predefined label space. In this project, the main task is binary classification between dogs and cats, which is a classical supervised closed-set recognition problem because both training and evaluation categories are known in advance. Under this setting, convolutional and transformer-based models are optimized with labeled samples by minimizing a classification loss and then evaluated on held-out images from the same label space.

The broader literature, however, spans several harder settings. In unsupervised or self-supervised learning, the model first learns generic visual representations without class labels and is then adapted to downstream tasks [8-10]. In open-set recognition, a test image may belong to an unseen class and the model must decide whether to reject it rather than force a wrong prediction; OpenMax is a representative early deep -learning solution to this problem [13]. Under domain shift, training and test data come from different distributions, which motivates domain-adversarial alignment and related transfer methods [12]. Finally, class imbalance causes the empirical risk to be dominated by frequent categories, and practical remedies include reweighting, resampling, and focal-style objectives.

1.2 Recent progress in image classification

The modern development of image classification has been driven by two complementary lines of work. The first line improves convolutional backbones. ResNet introduced residual learning and made substantially deeper CNNs trainable [1]. Later, MobileNetV3 optimized the accuracy-latency trade-off for efficient deployment, while ConvNeXt and ConvNeXt V2 showed that pure CNNs remain highly competitive when trained with modern design and pretraining strategies [2], [6-7].

The second line is transformer-centric. ViT demonstrated that image patches can be processed as a token sequence and, when pretrained at scale, achieve strong image-recognition performance [3]. DeiT reduced the data and hardware barrier of training ViTs, Swin introduced a hierarchical shifted-window design suitable for general vision backbones, and MAE/DINO-style self-supervised learning greatly strengthened representation quality [4], [5], [8], [10]. More recently, DINOv2 and InternImage highlighted the trend toward robust vision foundation models that transfer well across tasks and domains [9], [14].

1.3 Representative groups and works of interest

In recent years, two major research trajectories have emerged as particularly influential in the field of computer vision. The first is the Google Research/DeepMind line, exemplified by Vision Transformer (ViT) [3]. The significance of this direction lies in its systematic demonstration that transformer architectures, when trained on data at sufficient scale, can serve as highly competitive visual backbones, thereby challenging the long-standing dominance of convolutional inductive biases in vision tasks. Beyond establishing the feasibility of pure transformer-based vision models, ViT also stimulated extensive subsequent research on scaling laws, visual tokenization, training data regimes, and data-efficient learning strategies.

The second major trajectory is represented by the large-scale pretraining paradigm advanced by Meta AI / FAIR, including DINO, MAE, DINOv2, and CLIP-style transfer learning frameworks [8–11]. These works collectively show that self-supervised or weakly supervised pretraining can yield highly generalizable visual representations with strong transferability across a wide range of downstream tasks, including image classification, object detection, and semantic segmentation.

In addition, Microsoft Research has made equally important contributions through works such as Swin Transformer and ConvNeXt [5-6], which further advanced visual backbone design by improving the balance among representational power, computational efficiency, and practical deployability.

1.4 Baseline choice for this project

For the present project, a pretrained residual network is the most suitable baseline. The Dogs vs. Cats task is medium-scale, supervised, and visually well structured. Therefore, a model with strong generic features, stable convergence, and moderate computational demand is preferable to a very large transformer that requires heavy pretraining or expensive tuning. ResNet18 offers an excellent accuracy-efficiency trade-off, while ResNet34 provides a stronger but still practical upper baseline. MobileNetV3-Small is added as an efficiency-oriented comparison, and SmallCNN is included to show the gap between transfer learning and training a compact network from scratch on the same task.

Setting	Representative methods	Main idea	Relevance to this project
Supervised closed-set	ResNet [1], MobileNetV3 [2], ViT [3]	Learn a direct mapping from labeled images to known classes.	This is the main Dogs vs. Cats formulation.
Self-supervised / transfer	DINO [8], MAE [10], DINOv2 [9], CLIP [11]	Learn generic features first, then fine-tune or linear-probe on the target task.	Motivates using ImageNet-pretrained backbones.
Open-set recognition	OpenMax [13]	Reject unknown categories rather than forcing an in-distribution label.	Explains why confidence alone may be unreliable outside the label space.
Domain shift / adaptation	DANN [12]	Make learned features invariant across source and target domains.	Relevant when train/test imagery differ in style, source, or capture conditions.

Table 1. Literature Survey Summary across Common Image-classification Settings

2. Dataset, Pre-processing, and Reproducibility

2.1 Dogs vs. Cats dataset composition

According to the project brief, the provided Dogs vs. Cats dataset contains 20,000 training images, 5,000 validation images, and 500 unlabeled test images. The training and validation folders are class-balanced,

with separate dog and cat subdirectories, which makes the task suitable for plain cross-entropy optimization without any imbalance correction in the main experiment. Because the test split is unlabeled, it is used only for prediction export rather than for direct accuracy reporting.

Split	Cats	Dogs	Total
Training	10,000	10,000	20,000
Validation	2,500	2,500	5,000
Test	Unknown	Unknown	500 unlabeled images

Table 2. Dataset Summary for the Dogs vs. Cats Task

2.2 Pre-processing and augmentation pipeline

The training pipeline first resizes every image to 224 x 224 pixels, converts it to a tensor, and normalizes it with the standard ImageNet mean and standard deviation. The use of ImageNet statistics is appropriate because the pretrained backbones were originally optimized under the same normalization convention. For Dogs vs. Cats training only, the code additionally applies random horizontal flipping, random rotation within +/-15 degrees, and a light ColorJitter perturbation. This augmentation policy is deliberately moderate: it enlarges appearance diversity without strongly distorting semantic content.

Stage	Transform(s)	Purpose
Dogs vs. Cats train	Resize(224,224), RandomHorizontalFlip, RandomRotation(15°), ColorJitter, Normalize(ImageNet)	Improve robustness and reduce overfitting while staying visually plausible.
Dogs vs. Cats val/test	Resize(224,224), Normalize(ImageNet)	Keep evaluation deterministic and comparable.
CIFAR-10 train	Resize(224,224), RandomHorizontalFlip, RandomCrop(224,padding=12), Normalize(CIFAR-10)	Adapt small CIFAR images to pretrained backbones and preserve local variation.
CIFAR-10 val/test	Resize(224,224), Normalize(CIFAR-10)	Deterministic evaluation on held-out images.

Table 3. Data Transforms used in the Task

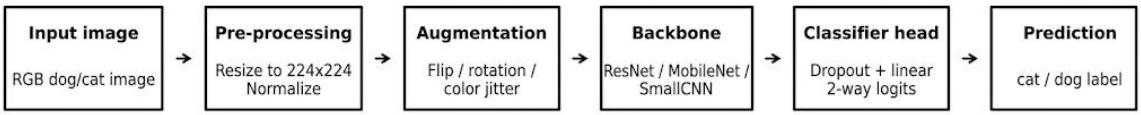


Figure 1. End-to-end Training and Inference Pipeline used for the Dogs vs. Cats

2.3 Reproducibility controls

The code fixes the random seed to 42, seeds Python, NumPy, and PyTorch, disables nondeterministic cuDNN benchmarking, and stores checkpoints for the best validation model. In addition, the test prediction script sorts image IDs before writing submission.csv, which ensures a stable submission format.

3. Model Design and Training Strategy

3.1 Tested model families

Four backbones are evaluated for Dogs vs. Cats. ResNet18 and ResNet34 replace the original fully connected layer with Dropout(0.2) followed by a new linear classifier for two classes. MobileNetV3-Small replaces the last classifier layer with a two-class linear head, providing a compact deployment-oriented alternative. SmallCNN is trained from scratch and consists of four convolutional stages with batch normalization and max pooling, followed by an adaptive average pooling layer and a two-layer multilayer perceptron classifier. Except for SmallCNN, the models use ImageNet-pretrained weights.

Model	Initialization	Classifier adaptation	Parameters	Role in experiments
ResNet18	ImageNet pretrained	Dropout(0.2) + Linear(512 -> 2)	11.18M	Main baseline; also reused for CIFAR-10 and imbalance study.
ResNet34	ImageNet pretrained	Dropout(0.2) + Linear(512 -> 2)	21.29M	Higher-capacity residual baseline.
MobileNet V3-Small	ImageNet pretrained	Replace final classifier layer with Linear(1024 -> 2)	1.52M	Efficiency-oriented comparison.
SmallCNN	Random initialization	Conv-BN-ReLU blocks + AdaptiveAvgPool + MLP head	0.42M	Lightweight model trained from scratch.

Table 4. Backbone Architectures Instantiated by the Main Code

3.2 Objective functions

For balanced Dogs vs. Cats training, the default objective is the standard cross-entropy loss. For imbalance experiments, the same codebase additionally supports class-weighted cross-entropy and focal loss. The weighted loss compensates minority classes through larger coefficients, whereas focal loss down-weights already easy examples and concentrates gradient mass on hard or rare samples.

$$L_{CE} = -\sum_{c=1}^{C} y_c \log(p_c)$$

$$L_{WCE} = -\sum_{c=1}^{C} \alpha_c y_c \log(p_c)$$

$$L_{Focal} = -\sum_{c=1}^{C} \alpha_c (1-p_c)^{\gamma} y_c \log(p_c)$$

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}(\hat{y}_i = y_i)$$

In the uploaded implementation, gamma is set to 2.0 for focal loss. WeightedRandomSampler is also supported; it modifies the data distribution seen by the optimizer by sampling training images with probability inversely proportional to their class frequency.

3.3 Optimization strategy

Dogs vs. Cats training uses Adam with learning rate 1×10^{-4}, weight decay 1e-4, batch size 32, and up to 12 epochs. The default scheduler is ReduceLROnPlateau, and early stopping with patience 4 terminates

training when validation accuracy stalls. For CIFAR-10, the script switches to a 15-epoch schedule with batch size 64 and cosine-annealing learning-rate decay. This design is sensible: the binary Dogs vs. Cats task converges very quickly under transfer learning, whereas CIFAR-10 benefits from a slightly longer and smoother training schedule.

Hyperparameter	Dogs vs. Cats	CIFAR-10 / imbalance
Image size	224 x 224	224 x 224
Optimizer	Adam	Adam
Learning rate	1e-4	1e-4
Weight decay	1e-4	1e-4 (CIFAR-10); none explicitly set in imbalance script
Batch size	32	64
Scheduler	ReduceLROnPlateau (default)	CosineAnnealingLR for CIFAR-10
Early stopping	Patience = 4	Not explicitly used in CIFAR-10 / imbalance scripts
Seed	42	42

Table 5. Main Hyperparameters

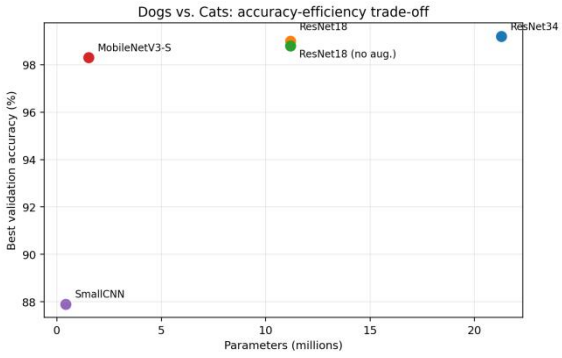
4. Experimental Results on Dogs vs. Cats

4.1 Quantitative comparison

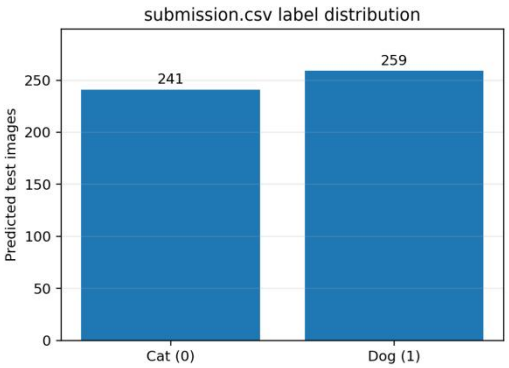
Table 6 summarizes the main Dogs vs. Cats results recoverable from the archived output figures. Among the tested backbones, ResNet34 achieves the strongest validation performance at approximately 99.2%, closely followed by ResNet18 at about 99.0%. MobileNetV3-Small is particularly attractive from an efficiency standpoint: despite having only about 1.52M parameters, it still reaches approximately 98.3% validation accuracy. By contrast, SmallCNN is substantially less stable and settles in the high-80% range, illustrating the advantage of transfer learning for this task.

Model	Parameters	Best val. acc.	Observation
ResNet34	21.29M	99.2%	Best overall accuracy; smooth convergence and low final loss.
ResNet18	11.18M	99.0%	Excellent trade-off between performance and computational cost.
ResNet18 (no augmentation)	11.18M	98.8%	Competitive, but less stable validation trajectory.
MobileNetV3-Small	1.52M	98.3%	Very parameter-efficient while remaining highly accurate.
SmallCNN	0.42M	87.9%	Lowest-capacity model; fluctuating optimization and weaker generalization.

Table 6. Dogs vs. Cats Validation Results



(a) Parameter count versus validation accuracy



(b) Predicted label distribution on the unlabeled test split

Figure 2. Accuracy-efficiency Analysis and Submission Statistics for Dogs vs. Cats

4.2 Training dynamics

The accuracy and loss curves indicate clear differences in optimization dynamics across models. Pretrained backbone networks, particularly the residual architectures, achieve rapid performance gains in the early training stage while maintaining low validation loss and a small generalization gap. This suggests that the generic visual representations learned during pretraining transfer effectively to the binary classification task, reducing the difficulty of feature learning and improving training stability.

In contrast, SmallCNN converges more slowly and exhibits stronger fluctuations throughout training, especially in the local variations of both accuracy and loss. This implies greater sensitivity to parameter initialization and mini-batch perturbations, as well as comparatively limited feature extraction and representation capacity. Without pretrained priors, the network must learn discriminative features entirely from scratch, making optimization less stable and convergence less sufficient.

Overall, the results show that deeper pretrained backbones offer clear advantages over a lightweight custom convolutional network in both optimization behaviour and generalization ability. For a limited-scale dataset with relatively clear class structure, transfer learning provides especially notable benefits, particularly in stable convergence and efficient feature reuse, which largely explains why the residual models consistently outperform SmallCNN.

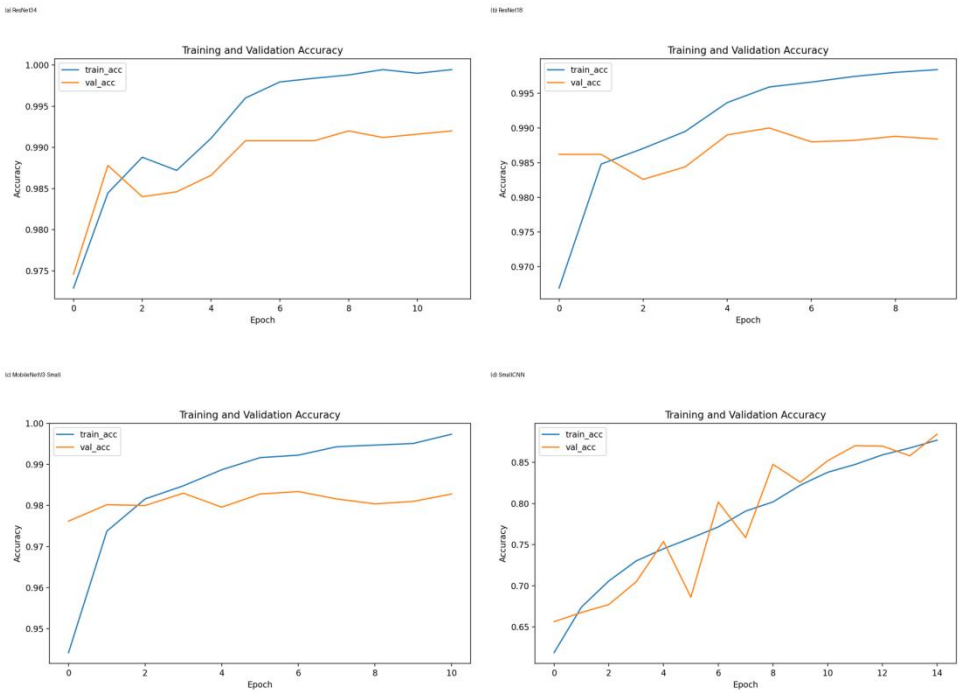


Figure 3. Accuracy Curves for the Main Dogs vs. Cats Backbones

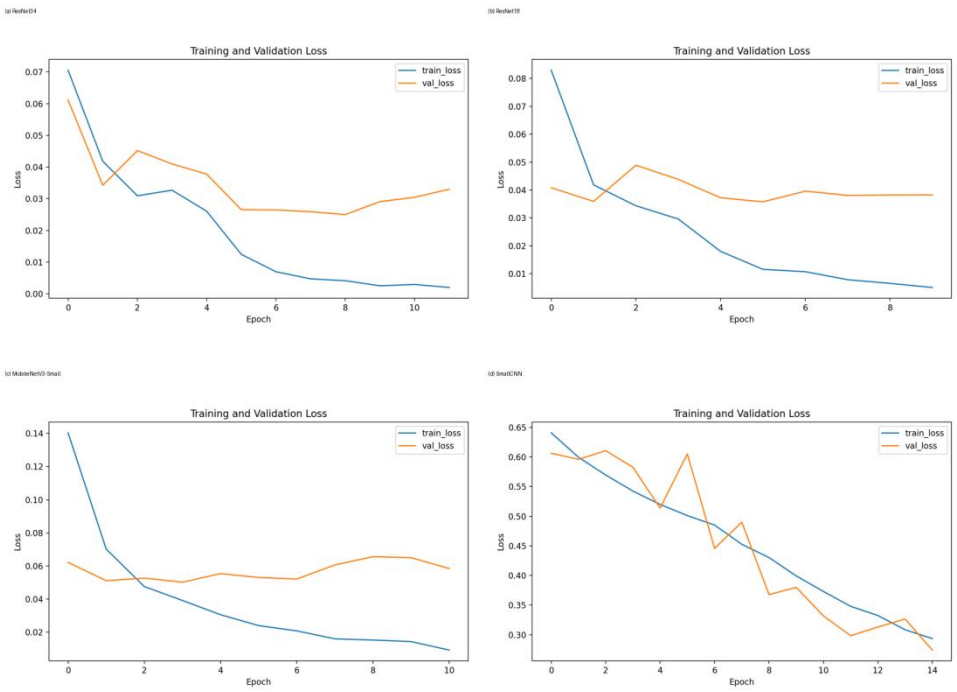


Figure 4. Loss Curves for the Main Dogs vs. Cats Backbones

4.3 Effect of augmentation

A controlled ablation is available through the ResNet18 and ResNet18-no-augmentation runs. Both variants reach strong validation accuracy, but the augmented run exhibits a more favourable loss profile and better mid-training stability. The non-augmented run shows a brief overfitting tendency: training accuracy keeps rising while validation accuracy softens for several epochs before partially recovering. This behaviour supports the use of modest augmentation even when the source dataset is already large.

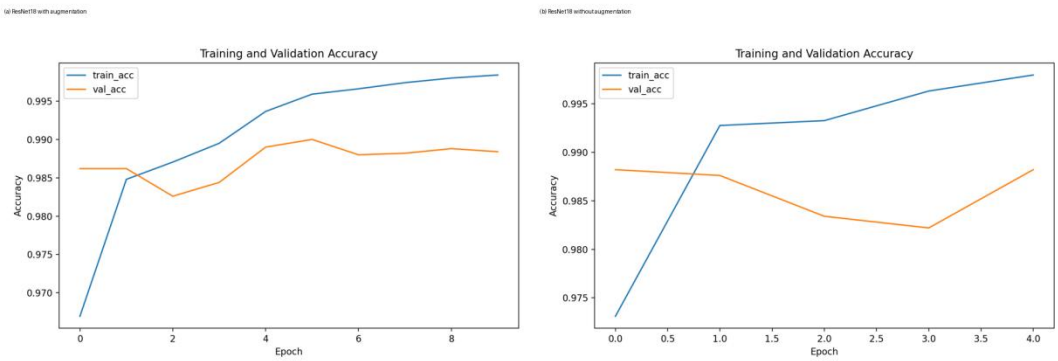


Figure 5. ResNet18 Accuracy with and without Training-time Augmentation

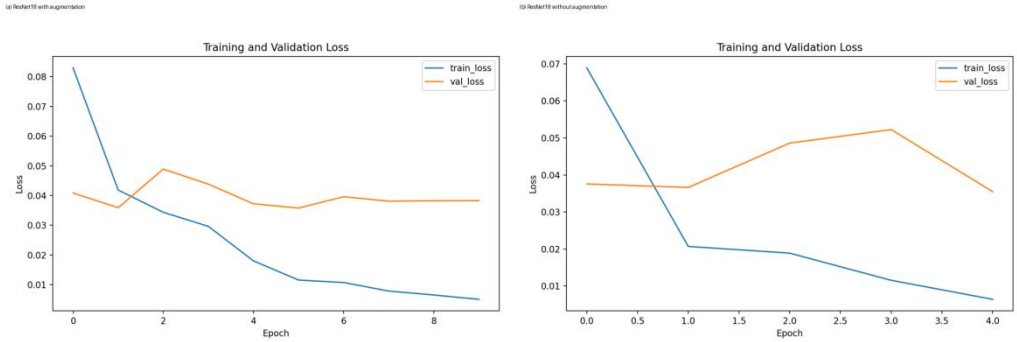


Figure 6. ResNet18 Loss with and without Training-time Augmentation

4.4 Test-set prediction export

The prediction script loads the best checkpoint, processes the 500 unlabeled test images in sorted ID order, and writes a two-column submission.csv with fields id and label. The archived file contains 241 cat predictions and 259 dog predictions, which is close to balanced and therefore does not indicate any obvious class-collapse failure. Because the test labels are hidden, this file is the correct project deliverable for the Dogs vs. Cats test set.

4.5 Strengths and likely failure modes

Based on our experimental results, combined with existing curves and model architecture, the following well-founded qualitative analysis can be conducted. Correct predictions are expected to come from images with a dominant foreground animal, clear texture or face cues, and limited background clutter. These cases match the strengths of pretrained CNNs: local edge and texture detectors, compositional mid-level features, and robust category templates learned from ImageNet.

Likely failure cases include unusual poses, partial occlusion, very low contrast, motion blur, and images where background context is stronger than the animal itself. Dog and cat breeds with atypical fur patterns can also compress the inter-class margin. In practice, such errors are consistent with the fact that even high-performing closed-set classifiers remain sensitive to object scale, crop quality, and spurious context. This is precisely why augmentation, better calibration, and richer pretraining remain important even on seemingly simple binary datasets.

5. Hyperparameter Discussion

The chosen hyperparameters are well aligned with the task. A learning rate of 1×10^{-4} is conservative enough for fine-tuning pretrained backbones without destroying transferred features, yet large enough for quick convergence on a simple binary problem. The image resolution of 224 x 224 follows common ImageNet-pretrained practice and avoids mismatches with backbone design. Batch size 32 for Dogs vs. Cats and 64 for CIFAR-10 balances throughput and memory usage. Weight decay 1e-4 provides mild regularization, while dropout in the classifier head reduces overfitting on the newly learned final mapping.

ResNet18 is the most balanced configuration because it already reaches near-saturated validation performance while remaining far lighter than ResNet34. ResNet34 slightly improves accuracy, but the marginal gain is much smaller than the increase in parameter count. MobileNetV3-Small is attractive when inference efficiency matters, whereas SmallCNN should be seen mainly as an educational baseline for understanding the advantage brought by transfer learning.

6. Extension to Multi-class Classification on CIFAR-10

6.1 Dataset adaptation and task changes

CIFAR-10 extends the project from binary classification to a 10-class recognition problem with 50,000 training images and 10,000 test images [15]. Compared with Dogs vs. Cats, the images are much smaller and visually more diverse, so the pipeline resizes them to 224 x 224 and replaces the final classifier with a 10-way output layer. The code randomly withholds 10% of the training set as validation data and evaluates on the official test set during training. This design allows direct reuse of the pretrained backbone while preserving a principled validation protocol.

6.2 Results and interpretation

The saved CIFAR-10 curves show that the fine-tuned ResNet18 reaches approximately 96.8% validation accuracy and converges cleanly within 15 epochs. It demonstrates that the selected baseline transfers effectively from binary pet classification to a broader object-recognition task. The training curve rises rapidly, while validation loss decreases and then stabilizes, indicating a healthy optimization regime with limited overfitting.

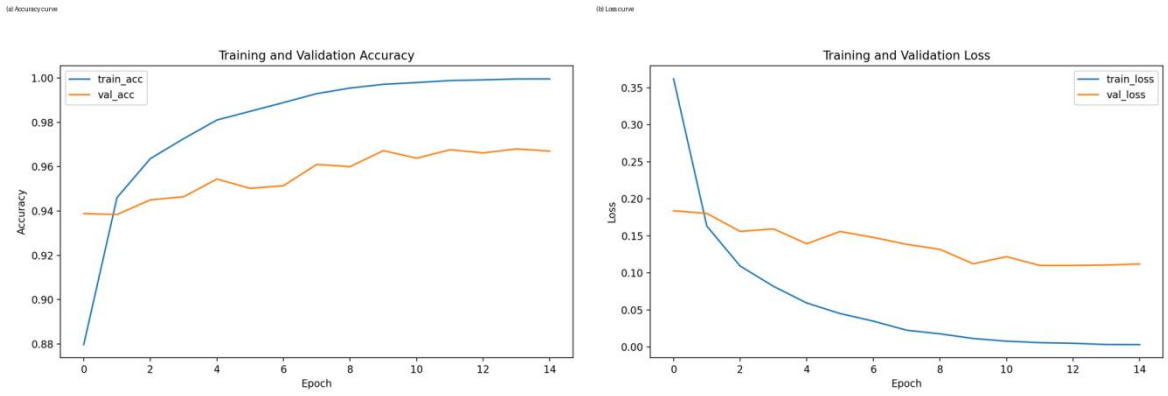


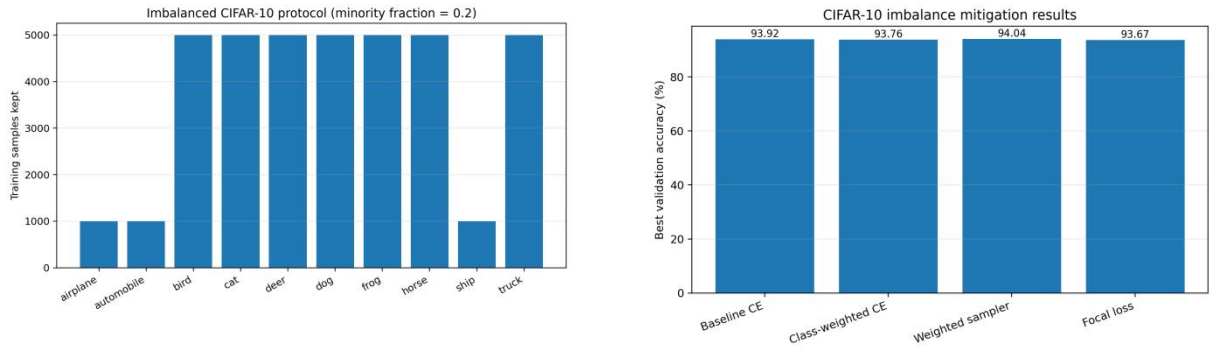
Figure 7. CIFAR-10 Training Curves for the ResNet18 Extension Experiment

Relative to Dogs vs. Cats, the CIFAR-10 task requires three main changes: a 10-class classifier head, CIFAR-specific normalization statistics, and slightly stronger spatial augmentation through random cropping with padding.

7. Handling Data Imbalance on CIFAR-10

7.1 Imbalance protocol

To simulate long-tail supervision, the uploaded imbalance experiment reduces classes 0, 1, and 8 to 20% of their original training size, leaving the remaining classes untouched. Since CIFAR-10 class indices 0, 1, and 8 correspond to airplane, automobile, and ship, these categories drop from 5,000 to 1,000 examples each, while all other classes remain at 5,000. This creates a meaningful but still recoverable imbalance scenario.



(a) Artificially imbalanced training distribution

(b) Validation accuracy under different remedies

Figure 8. CIFAR-10 Imbalance Protocol and Mitigation Results

7.2 Two practical remedies and an additional extension

The first remedy is class-weighted cross-entropy, where minority classes receive larger coefficients in the loss. This directly increases the penalty of misclassifying rare categories. The second remedy is WeightedRandomSampler, which changes the mini-batch composition so that minority-class examples are sampled more often during training. An additional extension is focal loss, which emphasizes hard samples by down-weighting easy ones. These three strategies are all reasonable and easy to integrate into the same codebase.

Method	Best val. acc.	Delta vs. baseline	Interpretation
Baseline cross-entropy	93.92%	0.00%	Reference system under skewed data.
Class-weighted cross-entropy	93.76%	-0.16%	Helps minority classes, but may slightly over-correct in this setup.
Weighted sampler	94.04%	+0.12%	Best result; improves effective exposure of minority examples.
Focal loss	93.67%	-0.25%	Reasonable, but not strongest for this particular imbalance level.

Table 7. Exact Results of the Imbalance Experiment

The weighted sampler achieves the best validation accuracy at 94.04%, slightly outperforming the baseline and the loss-based corrections. This outcome is plausible: when the imbalance is moderate rather than extreme, improving the effective mini-batch class balance can be more beneficial than aggressively reshaping the loss surface. In contrast, focal loss is often most useful when the training set contains a much larger share of trivial majority examples or many hard negatives.

8. Reproducibility and Running Instructions

The project is organized as a compact PyTorch codebase with separate scripts for Dogs vs. Cats training, test prediction, CIFAR-10 training, and imbalance analysis. The recommended workflow is to install the listed dependencies, train a Dogs vs. Cats backbone, export submission.csv for the unlabeled test set, and then run the CIFAR-10 experiments.

```
pip install -r requirements.txt
```

```
python train_dogs_vs_cats.py --data_root /path/to/dataset \
  --model_name resnet18 --epochs 12 --batch_size 32 --lr 1e-4 \
  --output_dir outputs/dogs_vs_cats_resnet18
```

```
python predict_test.py --test_dir /path/to/dataset/test \
  --checkpoint outputs/dogs_vs_cats_resnet18/best_model.pth \
  --model_name resnet18 --output_csv outputs/submission.csv
```

```
python train_cifar10.py --data_root ./data --model_name resnet18 \
  --epochs 15 --output_dir outputs/cifar10_resnet18
```

```
python imbalance_experiment.py --data_root ./data --epochs 8 \
  --minority_fraction 0.2 --minority_classes 0 1 8
```

Table 8. Command-Line Instructions for Reproducing the Main Experiments

9. Conclusion

This mini project demonstrates that ResNet34 provides the strongest archived validation result (~99.2%), while ResNet18 offers the best overall practicality and is therefore the most suitable default baseline. MobileNetV3-Small shows that strong performance is still achievable under tighter model-size constraints, whereas SmallCNN highlights the gap between scratch training and pretrained representations.

The same pipeline extends naturally to CIFAR-10, where a fine-tuned ResNet18 reaches approximately 96.8% validation accuracy in the saved run. Under artificially imbalanced supervision, weighted sampling proves to be the most effective of the tested corrections, reaching 94.04% validation accuracy. Taken together, these experiments support a clear high-level message: for medium-scale image classification,

strong pretraining, careful data processing, stable optimization, and simple imbalance-aware sampling already provide a highly competitive solution.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016.
[2] A. Howard, M. Sandler, G. Chu, et al., "Searching for MobileNetV3," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2019.
[3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in Int. Conf. Learn. Representations (ICLR), 2021.
[4] H. Touvron, M. Cord, M. Douze, et al., "Training Data-Efficient Image Transformers and Distillation through Attention," in Proc. Int. Conf. Mach. Learn. (ICML), 2021.
[5] Z. Liu, Y. Lin, Y. Cao, et al., "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2021.
[6] Z. Liu, H. Mao, C.-Y. Wu, et al., "A ConvNet for the 2020s," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022.
[7] S. Woo, S. Debnath, X. Hu, et al., "ConvNeXt V2: Co-Designing and Scaling ConvNets with Masked Autoencoders," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2023.
[8] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2021.
[9] M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning Robust Visual Features without Supervision," arXiv preprint arXiv:2304.07193, 2023.
[10] K. He, X. Chen, S. Xie, Y. Li, P. Dollar, and R. Girshick, "Masked Autoencoders Are Scalable Vision Learners," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022.
[11] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models From Natural Language Supervision," in Proc. Int. Conf. Mach. Learn. (ICML), 2021.
[12] Y. Ganin, E. Ustinova, H. Ajakan, et al., "Domain-Adversarial Training of Neural Networks," J. Mach. Learn. Res., vol. 17, no. 59, pp. 1-35, 2016.
[13] A. Bendale and T. Boult, "Towards Open Set Deep Networks," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016.
[14] W. Wang, W. Chen, Q. Qiu, et al., "InternImage: Exploring Large-Scale Vision Foundation Models with Deformable Convolutions," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2023.
[15] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," University of Toronto, Technical Report, 2009.

Appendix A. Core Code

(1) Model.py

```
from typing import Optional
import torch
import torch.nn as nn
from torchvision_compat import ensure_torchvision_compat
ensure_torchvision_compat()
from torchvision import models
class SmallCNN(nn.Module):
   def __init__(self, num_classes: int = 2, input_size: int = 224):
       super().__init__()
       self.features = nn.Sequential(
           nn.Conv2d(3, 32, kernel_size=3, padding=1),
           nn.BatchNorm2d(32),
           nn.ReLU(inplace=True),
           nn.MaxPool2d(2),

           nn.Conv2d(32, 64, kernel_size=3, padding=1),
           nn.BatchNorm2d(64),
           nn.ReLU(inplace=True),
           nn.MaxPool2d(2),

           nn.Conv2d(64, 128, kernel_size=3, padding=1),
           nn.BatchNorm2d(128),
           nn.ReLU(inplace=True),
           nn.MaxPool2d(2),
```

```
            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.AdaptiveAvgPool2d((1, 1)),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Dropout(0.3),
            nn.Linear(256, 128),
            nn.ReLU(inplace=True),
            nn.Dropout(0.2),
            nn.Linear(128, num_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.classifier(x)
        return x

class FocalLoss(nn.Module):
    def __init__(self, alpha: Optional[torch.Tensor] = None, gamma: float = 2.0,
reduction: str = 'mean'):
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.reduction = reduction

    def forward(self, logits: torch.Tensor, targets: torch.Tensor) ->
torch.Tensor:
        ce_loss = nn.functional.cross_entropy(logits, targets, weight=self.alpha,
reduction='none')
        pt = torch.exp(-ce_loss)
        loss = ((1 - pt) ** self.gamma) * ce_loss
        if self.reduction == 'mean':
            return loss.mean()
        if self.reduction == 'sum':
            return loss.sum()
        return loss

def build_model(model_name: str = 'resnet18', num_classes: int = 2, pretrained:
bool = True, freeze_backbone: bool = False):
    model_name = model_name.lower()

    if model_name == 'resnet18':
        weights = models.ResNet18_Weights.DEFAULT if pretrained else None
        model = models.resnet18(weights=weights)
        in_features = model.fc.in_features
        model.fc = nn.Sequential(
            nn.Dropout(0.2),
            nn.Linear(in_features, num_classes)
        )
    elif model_name == 'resnet34':
        weights = models.ResNet34_Weights.DEFAULT if pretrained else None
        model = models.resnet34(weights=weights)
        in_features = model.fc.in_features
        model.fc = nn.Sequential(
```

```
            nn.Dropout(0.2),
            nn.Linear(in_features, num_classes)
        )
    elif model_name == 'mobilenet_v3_small':
        weights = models.MobileNet_V3_Small_Weights.DEFAULT if pretrained else
None
        model = models.mobilenet_v3_small(weights=weights)
        in_features = model.classifier[-1].in_features
        model.classifier[-1] = nn.Linear(in_features, num_classes)
    elif model_name == 'smallcnn':
        model = SmallCNN(num_classes=num_classes)
    else:
        raise ValueError(f'Unsupported model_name: {model_name}')

    if freeze_backbone and model_name != 'smallcnn':
        for param in model.parameters():
            param.requires_grad = False
        # unfreeze classifier head
        if hasattr(model, 'fc'):
            for param in model.fc.parameters():
                param.requires_grad = True
        if hasattr(model, 'classifier'):
            for param in model.classifier.parameters():
                param.requires_grad = True

    return model
```

(2) dataset.py

```
from pathlib import Path
from typing import List, Optional, Tuple
from PIL import Image
import torch
from torch.utils.data import Dataset
from torchvision_compat import ensure_torchvision_compat
ensure_torchvision_compat()
from torchvision import datasets, transforms

IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]
def build_train_transform(image_size: int = 224):
    return transforms.Compose([
        transforms.Resize((image_size, image_size)),
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomRotation(degrees=15),
        transforms.ColorJitter(brightness=0.15, contrast=0.15, saturation=0.1,
hue=0.02),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
        # transforms.Resize((image_size, image_size)),
        # transforms.ToTensor(),
        # transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])

def build_eval_transform(image_size: int = 224):
    return transforms.Compose([
        transforms.Resize((image_size, image_size)),
```

```
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])

class TestImageDataset(Dataset):
    def __init__(self, root_dir: str, transform=None):
        self.root_dir = Path(root_dir)
        self.transform = transform
        self.image_paths = sorted(
            [p for p in self.root_dir.iterdir() if p.suffix.lower() in {'.jpg',
'.jpeg', '.png', '.bmp'}],
            key=lambda x: int(x.stem) if x.stem.isdigit() else x.stem
        )

    def __len__(self) -> int:
        return len(self.image_paths)

    def __getitem__(self, index: int):
        image_path = self.image_paths[index]
        image = Image.open(image_path).convert('RGB')
        if self.transform is not None:
            image = self.transform(image)
        image_id = int(image_path.stem) if image_path.stem.isdigit() else
image_path.stem
        return image, image_id

def get_imagefolder_datasets(train_dir: str, val_dir: str, image_size: int =
224):
    train_dataset = datasets.ImageFolder(train_dir,
transform=build_train_transform(image_size))
    val_dataset = datasets.ImageFolder(val_dir,
transform=build_eval_transform(image_size))
    return train_dataset, val_dataset

def get_class_weights_from_imagefolder(dataset: datasets.ImageFolder) ->
torch.Tensor:
    targets = [label for _, label in dataset.samples]
    num_classes = len(dataset.classes)
    counts = torch.bincount(torch.tensor(targets), minlength=num_classes).float()
    weights = counts.sum() / (num_classes * counts.clamp(min=1.0))
    return weights

def get_sample_weights_from_imagefolder(dataset: datasets.ImageFolder) ->
List[float]:
    targets = [label for _, label in dataset.samples]
    counts = torch.bincount(torch.tensor(targets)).float()
    class_weights = 1.0 / counts.clamp(min=1.0)
    sample_weights = [class_weights[t].item() for t in targets]
    return sample_weights
```

(3) Train_dogs_vs_cats.py

```
import argparse
from pathlib import Path
import torch
import torch.nn as nn
from torch.optim import Adam
```

```
from torch.optim.lr_scheduler import ReduceLROnPlateau, StepLR
from torch.utils.data import DataLoader, WeightedRandomSampler
from tqdm import tqdm
from dataset import get_class_weights_from_imagefolder,
get_imagefolder_datasets, get_sample_weights_from_imagefolder
from models import FocalLoss, build_model
from utils import EarlyStopping, compute_accuracy, ensure_dir, get_device,
load_checkpoint, plot_history, save_checkpoint, save_json, seed_everything,
summarize_classification

def parse_args():
   parser = argparse.ArgumentParser(description='Train Dogs vs Cats classifier')
   parser.add_argument('--data_root', type=str, required=True, help='Dataset
root containing train/, val/, test/')
   parser.add_argument('--model_name', type=str, default='resnet18',
choices=['resnet18', 'resnet34', 'mobilenet_v3_small', 'smallcnn'])
   parser.add_argument('--image_size', type=int, default=224)
   parser.add_argument('--epochs', type=int, default=12)
   parser.add_argument('--batch_size', type=int, default=32)
   parser.add_argument('--lr', type=float, default=1e-4)
   parser.add_argument('--weight_decay', type=float, default=1e-4)
   parser.add_argument('--num_workers', type=int, default=2)
   parser.add_argument('--seed', type=int, default=42)
   parser.add_argument('--output_dir', type=str, default='outputs/dogs_vs_cats')
   parser.add_argument('--scheduler', type=str, default='plateau',
choices=['plateau', 'step', 'none'])
   parser.add_argument('--step_size', type=int, default=5)
   parser.add_argument('--gamma', type=float, default=0.1)
   parser.add_argument('--freeze_backbone', action='store_true')
   parser.add_argument('--use_weighted_sampler', action='store_true')
   parser.add_argument('--use_class_weights', action='store_true')
   parser.add_argument('--use_focal_loss', action='store_true')
   parser.add_argument('--early_stop_patience', type=int, default=4)
   parser.add_argument('--resume', type=str, default='')
   return parser.parse_args()

def train_one_epoch(model, loader, criterion, optimizer, device):
   model.train()
   running_loss = 0.0
   preds_all, labels_all = [], []

   for images, labels in tqdm(loader, desc='Train', leave=False):
       images = images.to(device)
       labels = labels.to(device)

       optimizer.zero_grad()
       logits = model(images)
       loss = criterion(logits, labels)
       loss.backward()
       optimizer.step()

       running_loss += loss.item() * images.size(0)
       preds = logits.argmax(dim=1)
       preds_all.extend(preds.detach().cpu().tolist())
       labels_all.extend(labels.detach().cpu().tolist())

   epoch_loss = running_loss / len(loader.dataset)
```

```
    epoch_acc = compute_accuracy(labels_all, preds_all)
    return epoch_loss, epoch_acc

@torch.no_grad()
def evaluate(model, loader, criterion, device, class_names):
    model.eval()
    running_loss = 0.0
    preds_all, labels_all = [], []

    for images, labels in tqdm(loader, desc='Val', leave=False):
        images = images.to(device)
        labels = labels.to(device)
        logits = model(images)
        loss = criterion(logits, labels)

        running_loss += loss.item() * images.size(0)
        preds = logits.argmax(dim=1)
        preds_all.extend(preds.detach().cpu().tolist())
        labels_all.extend(labels.detach().cpu().tolist())

    epoch_loss = running_loss / len(loader.dataset)
    epoch_acc = compute_accuracy(labels_all, preds_all)
    summary = summarize_classification(labels_all, preds_all, class_names)
    return epoch_loss, epoch_acc, summary

def main():
    args = parse_args()
    seed_everything(args.seed)
    device = get_device()
    ensure_dir(args.output_dir)

    train_dir = str(Path(args.data_root) / 'train')
    val_dir = str(Path(args.data_root) / 'val')

    train_dataset, val_dataset = get_imagefolder_datasets(train_dir, val_dir,
args.image_size)

    if args.use_weighted_sampler:
        sample_weights = get_sample_weights_from_imagefolder(train_dataset)
        sampler = WeightedRandomSampler(weights=sample_weights,
num_samples=len(sample_weights), replacement=True)
        train_loader = DataLoader(train_dataset, batch_size=args.batch_size,
sampler=sampler, num_workers=args.num_workers)
    else:
        train_loader = DataLoader(train_dataset, batch_size=args.batch_size,
shuffle=True, num_workers=args.num_workers)

    val_loader = DataLoader(val_dataset, batch_size=args.batch_size,
shuffle=False, num_workers=args.num_workers)

    model = build_model(args.model_name, num_classes=2, pretrained=True,
freeze_backbone=args.freeze_backbone).to(device)

    class_weights = None
    if args.use_class_weights:
        class_weights =
get_class_weights_from_imagefolder(train_dataset).to(device)
```

```
    if args.use_focal_loss:
        criterion = FocalLoss(alpha=class_weights, gamma=2.0)
    else:
        criterion = nn.CrossEntropyLoss(weight=class_weights)

    optimizer = Adam(filter(lambda p: p.requires_grad, model.parameters()),
lr=args.lr, weight_decay=args.weight_decay)

    if args.scheduler == 'plateau':
        scheduler = ReduceLROnPlateau(optimizer, mode='max', factor=args.gamma,
patience=2)
    elif args.scheduler == 'step':
        scheduler = StepLR(optimizer, step_size=args.step_size, gamma=args.gamma)
    else:
        scheduler = None

    start_epoch = 0
    best_val_acc = 0.0
    history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}
    early_stopping = EarlyStopping(patience=args.early_stop_patience, mode='max')

    if args.resume:
        ckpt = load_checkpoint(args.resume, model, optimizer=optimizer,
map_location=device)
        start_epoch = ckpt.get('epoch', 0) + 1
        best_val_acc = ckpt.get('best_val_acc', 0.0)
        history = ckpt.get('history', history)

    class_names = train_dataset.classes
    print(f'Using device: {device}')
    print(f'Classes: {class_names}')

    for epoch in range(start_epoch, args.epochs):
        print(f'\nEpoch {epoch + 1}/{args.epochs}')
        train_loss, train_acc = train_one_epoch(model, train_loader, criterion,
optimizer, device)
        val_loss, val_acc, val_summary = evaluate(model, val_loader, criterion,
device, class_names)

        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_acc)

        print(f'train_loss={train_loss:.4f}, train_acc={train_acc:.4f},
val_loss={val_loss:.4f}, val_acc={val_acc:.4f}')

        improved = early_stopping.step(val_acc)
        if improved and val_acc >= best_val_acc:
            best_val_acc = val_acc
            save_checkpoint({
                'epoch': epoch,
                'best_val_acc': best_val_acc,
                'model_state_dict': model.state_dict(),
                'optimizer_state_dict': optimizer.state_dict(),
                'history': history,
```

```
                'class_names': class_names,
                'args': vars(args),
            }, str(Path(args.output_dir) / 'best_model.pth'))
            save_json(val_summary, str(Path(args.output_dir) /
'best_val_metrics.json'))

        if scheduler is not None:
            if args.scheduler == 'plateau':
                scheduler.step(val_acc)
            else:
                scheduler.step()

        if early_stopping.should_stop:
            print('Early stopping triggered.')
            break
    plot_history(history, str(Path(args.output_dir) / 'history.json'))
    save_json(history, str(Path(args.output_dir) / 'history.json'))
    print(f'Best validation accuracy: {best_val_acc:.4f}')
    print(f'Artifacts saved to: {args.output_dir}')

if __name__ == '__main__':
    main()
```